

Apostila do Laboratório de Linguagens de Programação

Módulo C++: Programação Orientada a Objetos

Pedro Henrique Ribeiro Dias

17 de setembro de 2026

Sumário

1	Relatório 6: Programação Orientada a Objetos em C++	2
1.1	Fundamentos Teóricos e Pilares da POO	2
1.1.1	A Mudança de Paradigma: Da Programação Estruturada à Orientação a Objetos	2
1.1.2	Os Blocos Fundamentais: Classes, Objetos, Estado e Comportamento	2
1.1.3	Os Quatro Pilares da Orientação a Objetos	3
1.2	Documentação Sintática das Estruturas de POO	5
1.2.1	Declaração de Classe e Modificadores de Acesso	5
1.2.2	Construtores e Lista de Inicialização	5
1.2.3	Herança Pública	5
1.2.4	Polimorfismo Dinâmico: <code>virtual</code> e <code>override</code>	5
1.2.5	Destrutor Virtual (<code>virtual ~Destrutor()</code>)	6
1.2.6	Ponteiros de Classe Base e Gerenciamento Dinâmico de Memória	6
1.3	Código de Referência Completo	6
1.4	Exercícios Práticos do Módulo	8

Capítulo 1

Relatório 6: Programação Orientada a Objetos em C++

1.1 Fundamentos Teóricos e Pilares da POO

1.1.1 A Mudança de Paradigma: Da Programação Estruturada à Orientação a Objetos

Para compreender a Programação Orientada a Objetos (POO), é fundamental entender a evolução na forma de estruturar o raciocínio computacional. Na programação estruturada ou procedural, o código é organizado como uma sequência linear ou hierárquica de procedimentos e rotinas que operam sobre dados armazenados em estruturas separadas. Nesse modelo, existe uma separação clara entre "quem executa a ação" (a função) e "quem sofre a ação" (os dados).

Conforme os sistemas de software crescem em escala e complexidade, essa separação gera desafios significativos de manutenção. Alterar a estrutura de um dado pode exigir a modificação de dezenas de funções espalhadas pelo código, aumentando o risco de efeitos colaterais indesejados.

A POO propõe uma mudança fundamental de perspectiva: em vez de tratar dados e funções como entidades separadas, ela os unifica em estruturas coesas e autônomas. O foco da modelagem deixa de ser a sequência de passos executados pelo computador e passa a ser as **entidades do domínio do problema** e as interações entre elas.

1.1.2 Os Blocos Fundamentais: Classes, Objetos, Estado e Comportamento

Antes de analisar os pilares da POO, é necessário dominar os quatro conceitos estruturais que sustentam esse paradigma:

- **Classe (O Modelo Conceitual):** Uma classe é uma especificação abstrata, funcionando como um projeto ou molde. Ela não ocupa espaço operacional como uma entidade ativa em memória; em vez disso, define a estrutura lógica, as propriedades que os elementos desse tipo possuirão e as ações que poderão executar. Classes podem abstrair conceitos do mundo real para o código, por exemplo, podemos criar uma classe *Animal*, *Pessoa*, *Carro*, *ContaBancária* e muito mais, portanto não se prendam ao modelar classes.
- **Objeto (A Entidade Concreta):** Um objeto é a concretização prática de uma classe — processo conhecido como *instanciação*. Se a classe é a planta arquitetônica de uma casa, o objeto é a casa física efetivamente construída na memória do computador durante a execução do programa. Um programa em POO consiste em um conjunto de objetos interagindo por meio do envio de mensagens. Continuando o exemplo dado no final da classe, a instância de um *Animal* poderia ser um cachorro real, por exemplo o Scooby-doo (não confundam estes conceitos, poderíamos criar uma classe *Cachorro*, uma abstração

deste animal para o código, e o objeto criado, a instância dela seria o cachorro Scooby-doo!)

- **Atributos e Estado:** Os atributos são as variáveis internas pertencentes a um objeto que armazenam suas características ou dados. O conjunto de valores armazenados nesses atributos em um determinado instante do tempo define o **estado** do objeto. O estado de um objeto é dinâmico e pode se alterar ao longo do ciclo de vida da aplicação. Por exemplo, mantendo a linha de Animais, estes podem ter espécie, nome e diversas outras características expressas em seus atributos.
- **Métodos e Comportamento:** Os métodos são as rotinas ou funções associadas a uma classe que operam sobre os seus atributos. Eles representam o **comportamento** do objeto, definindo as ações que ele pode realizar, as transformações que pode aplicar ao seu próprio estado ou as informações que pode fornecer a outros objetos. Por exemplo, mais uma vez em animais, um comportamento para um cachorro poderia ser andar(), latir(), rosnar() e tudo mais que fizer sentido para quem estiver modelando a classe.

1.1.3 Os Quatro Pilares da Orientação a Objetos

A POO se sustenta sobre quatro pilares teóricos fundamentais. A aplicação conjunta desses pilares permite a construção de softwares mais modulares, flexíveis, reutilizáveis e fáceis de manter.

1. Abstração

A abstração é o processo mental de isolar os aspectos essenciais de uma entidade do mundo real ou do domínio do problema, ignorando os detalhes irrelevantes para o contexto do sistema.

No desenvolvimento de software, não é viável nem necessário representar uma entidade em sua totalidade física ou biológica. A abstração estabelece o **fronteira do contexto**:

- Em um sistema acadêmico, a abstração de uma entidade "Pessoa" focará em atributos como matrícula, histórico de notas e curso.
- Em um sistema hospitalar, a mesma entidade "Pessoa" será abstraída focando em tipo sanguíneo, histórico médico e alergias.

A abstração correta garante que as classes contenham apenas o que é estritamente necessário para que o sistema cumpra seu propósito.

2. Encapsulamento

O encapsulamento é o mecanismo que agrupa dados (atributos) e procedimentos (métodos) em uma única unidade (o objeto) e **restringe o acesso direto** ao estado interno do objeto a partir do ambiente externo.

Os objetivos centrais do encapsulamento são:

- **Proteção de Dados:** Evitar que o estado interno do objeto seja alterado para valores inválidos ou incoerentes por agentes externos.
- **Ocultação da Implementação:** Permitir que a lógica interna de um método seja alterada pelo desenvolvedor sem impactar o restante do sistema, desde que a forma de interagir com ele continue a mesma.

O encapsulamento é operacionalizado por meio de **modificadores de acesso** (ou níveis de visibilidade):

- *Privado (Private)*: Os membros só podem ser acessados ou alterados por métodos da própria classe.
- *Público (Public)*: Os membros compõem a *interface pública* do objeto e podem ser acessados por qualquer outra parte do sistema.
- *Protegido (Protected)*: Os membros são acessíveis pela própria classe e por suas subclasses derivadas.

A comunicação segura com um objeto encapsulado ocorre mediante uma interface pública controlada, composta por métodos de validação, consulta e alteração.

3. Herança

A herança é um mecanismo de associação hierárquica que permite criar novas classes a partir de classes já existentes. Ela estabelece uma relação semântica do tipo "**é um**" ou "**é uma espécie de**".

Nessa estrutura:

- **Classe Base (Superclasse ou Classe Mãe)**: Define as características e comportamentos genéricos compartilhados por uma família de entidades.
- **Classe Derivada (Subclasse ou Classe Filha)**: Herda automaticamente a estrutura e o comportamento da classe base, podendo adicionar novos atributos/métodos específicos ou redefinir comportamentos herdados.

O principal benefício da herança é a **reutilização de código** e a organização hierárquica do conhecimento no sistema. Quando um comportamento comum a várias classes precisa ser alterado, a modificação pode ser feita centralizadamente na classe base.

4. Polimorfismo

Originado do grego "muitas formas", o polimorfismo é a capacidade de objetos de diferentes classes derivadas responderem à mesma mensagem (chamada de método) de maneiras distintas e adequadas à sua especificidade.

Para que o polimorfismo ocorra, é necessário haver uma interface comum (proporcionada por herança ou por contratos abstratos). O sistema pode tratar um grupo heterogêneo de objetos derivados como se fossem do tipo genérico da classe base, mas, no momento da execução, o comportamento individual de cada um será acionado.

Existem duas formas principais de manifestação conceitual:

- **Sobrescrita (Overriding)**: A subclasse reescreve a implementação de um método herdado da classe base para adaptá-lo ao seu comportamento específico. A decisão de qual método executar ocorre dinamicamente em tempo de execução (*late binding*).
- **Sobrecarga (Overloading)**: Uma mesma classe possui métodos com o mesmo nome, porém com assinaturas (parâmetros e tipos) diferentes. A escolha da execução é resolvida em tempo de compilação.

O polimorfismo reduz o acoplamento do código, pois permite escrever rotinas genéricas que funcionam com novos tipos de objetos sem a necessidade de modificar as estruturas condicionais do programa.

1.2 Documentação Sintática das Estruturas de POO

Esta seção detalha todas as palavras-chave e construções sintáticas utilizadas na implementação de POO em C++.

1.2.1 Declaração de Classe e Modificadores de Acesso

Uma classe é declarada com a palavra-chave `class`. O corpo da classe define os níveis de acesso aos seus membros:

```
1 class SerVivo {
2 protected:
3     std::string nome; // Acess vel pela classe e por classes derivadas
4     (filhas)
5 public:
6     // Acess vel de qualquer ponto do programa
7 };
```

- `private` (Padrão): Membros são acessíveis apenas pelos métodos da própria classe.
- `protected`: Membros são inacessíveis externamente, mas acessíveis dentro da própria classe e por suas subclasses derivadas.
- `public`: Membros podem ser acessados diretamente por qualquer objeto ou função do sistema.

1.2.2 Construtores e Lista de Inicialização

O construtor possui o mesmo nome da classe e é executado automaticamente no momento da instanciação. A sintaxe de **lista de inicialização de membros** (*Member Initializer List*) atribui valores aos membros antes da execução do corpo do construtor:

```
1 // Sintaxe: Construtor(parametros) : atributo(parametro) {}
2 SerVivo(string n) : nome(n) {}
```

1.2.3 Herança Pública

A herança estabelece uma relação do tipo "É UM". Para herdar publicamente de uma classe base:

```
1 class Humano : public SerVivo {
2 public:
3     // Repassa o par metro 'n' para o construtor da classe base SerVivo
4     Humano(string n) : SerVivo(n) {}
5 };
```

1.2.4 Polimorfismo Dinâmico: virtual e override

- `virtual` (na classe base): Declara um método que pode ser sobrescrito nas classes derivadas. Habilita o *dynamic dispatch* (ligação tardia em tempo de execução).
- `override` (na classe derivada): Garante em tempo de compilação que o método está sobrescrevendo exatamente uma função virtual da classe base.

```

1 // Na Classe Base:
2 virtual void apresentar() { ... }
3
4 // Na Classe Derivada:
5 void apresentar() override { ... }

```

1.2.5 Destrutor Virtual (virtual ~Destrutor())

Sempre que uma classe possui métodos virtuais e é manipulada por meio de ponteiros da classe base, seu destrutor **deve ser virtual**. Isso garante que o destrutor correto da classe derivada seja invocado na desalocação de memória.

```

1 virtual ~Servivo() {}

```

1.2.6 Ponteiros de Classe Base e Gerenciamento Dinâmico de Memória

O polimorfismo dinâmico exige o uso de ponteiros ou referências da classe base armazenando o endereço de instâncias das classes filhas alocadas dinamicamente via operador **new**.

```

1 // Armazenamento genérico via ponteiro da classe base
2 vector<Servivo*> seres;
3
4 // Instancia o dinâmico
5 seres.push_back(new Humano("Arthur"));
6
7 // Chamada polimórfica via ponteiro (operador ->)
8 for (Servivo* s : seres) {
9     s->apresentar();
10 }
11
12 // Libera a obrigatoriedade da memória alocada dinamicamente
13 for (Servivo* s : seres) {
14     delete s;
15 }

```

1.3 Código de Referência Completo

Abaixo está a implementação de referência utilizada para exemplificação do paradigma POO em C++:

```

1 #include <iostream>
2 #include <string>
3 #include <vector>
4 using namespace std;
5
6 // CLASSE BASE (Abstrato)
7 class Servivo {
8 protected:
9     string nome; // Encapsulamento protegido
10
11 public:
12     // Construtor com Lista de Inicialização
13     Servivo(string n) : nome(n) {}
14
15     // Método Virtual para Polimorfismo
16     virtual void apresentar() {

```

```

17         cout << "Eu sou um ser vivo chamado " << nome << "." << endl;
18     }
19
20     // Destrutor Virtual
21     virtual ~Servivo() {}
22 };
23
24 // HERAN A: Humano herda de Servivo
25 class Humano : public Servivo {
26 public:
27     Humano(string n) : Servivo(n) {}
28
29     // SOBRESCRITA (Override)
30     void apresentar() override {
31         cout << "Ol , eu sou o humano " << nome << ", mestre das
civiliza es." << endl;
32     }
33 };
34
35 class Elfo : public Servivo {
36 public:
37     Elfo(string n) : Servivo(n) {}
38
39     void apresentar() override {
40         cout << "Sauda es , eu sou o elfo " << nome << ", guardi o
das florestas." << endl;
41     }
42 };
43
44 class Fada : public Servivo {
45 public:
46     Fada(string n) : Servivo(n) {}
47
48     void apresentar() override {
49         cout << "Encantado! Eu sou a fada " << nome << ", portadora da
magia." << endl;
50     }
51 };
52
53 int main() {
54     // Vetor de ponteiros para a classe base (Polimorfismo)
55     vector<Servivo*> seres;
56
57     // Aloca o din mica de mem ria
58     seres.push_back(new Humano("Arthur"));
59     seres.push_back(new Elfo("Legolas"));
60     seres.push_back(new Fada("Luna"));
61     seres.push_back(new Servivo("Pedro"));
62
63     cout << "=== Apresenta es no mundo de fantasia ===" << endl;
64
65     // Execu o polim rfica
66     for (Servivo* s : seres) {
67         s->apresentar();
68     }
69
70     // Desaloca o de mem ria (Good Practices)
71     for (Servivo* s : seres) {

```

```

72     delete s;
73 }
74
75     return 0;
76 }

```

Listing 1.1: Exemplo de Aplicação de POO em C++

1.4 Exercícios Práticos do Módulo

Exercício 1: Abstração e Encapsulamento (Sistema de Combate de Robôs)

Crie uma classe `Robo` contendo os atributos `modelo` (string), `versao` (int), `potenciaLaser` (float) e `integridade` (int). Implemente o método `disparar(Robo &alvo)` que reduz a integridade do alvo com base na potência do laser do atacante. Instancie dois robôs e simule um confronto.

Exercício 2: Herança e Modificadores de Acesso (Persona RPG)

Implemente a classe base `Pessoa` com atributos privados `nome` e `idade`. Crie as classes derivadas `Protagonista` (com o atributo `nivel`) e `Personagem` (com o atributo `rank`). Demonstre o acesso correto aos membros através dos construtores da classe pai.

Exercício 3: Polimorfismo Dinâmico (Membros do Inatel)

Crie a classe base `MembroInatel` com o método virtual `identificar()`. Derive as classes `Coordenador` (atributo `departamento`) e `Pesquisador` (atributo `laboratorio`). Utilize ponteiros da classe base para chamar o método `identificar()` de forma polimórfica.

Exercício 4: Polimorfismo em Coleções (Conselho das Terras Ancestrais)

Crie a classe base `MembroConselho` e as derivadas `Anao`, `Orc` e `Draconato`. Armazene instâncias dessas classes em um `std::vector<MembroConselho*>` e percorra a coleção invocando o método sobrescrito `saudar()`.